

Program Analysis 2025-26

Lecture #19

Control Flow Analysis



UNIVERSITÀ
DI PISA

Existence

Existence of solutions

- So far we have only considered a procedure for validating whether or not a proposed solution $(\hat{C}, \hat{\rho})$ is acceptable
- Now it is natural to ask the following questions:
 - Does each expression e admit a Control Flow Analysis, i.e., does there **exist** $(\hat{C}, \hat{\rho})$ such that $(\hat{C}, \hat{\rho}) \models e$? **Yes**
- There may be **different possible analyses** for the same expression e
 - Does each expression e have a "least" CFA? **Yes**

Comparing analyses

Fewer possible flows $\Rightarrow$ more precise analysis

Which **partial order** to compare different CFAs?

The set of proposed solutions can be partially ordered by setting

pointwise inclusion

$$(\hat{C}_1, \hat{\rho}_1) \sqsubseteq (\hat{C}_2, \hat{\rho}_2) \text{ iff } \begin{cases} \forall \ell \in \mathbf{Lab} : \hat{C}_1(\ell) \subseteq \hat{C}_2(\ell) \\ \forall x \in \mathbf{Var} : \hat{\rho}_1(x) \subseteq \hat{\rho}_2(x) \end{cases}$$

smaller = more precise

This suffices for making the set of proposed solutions into a **complete lattice**

Comparing analyses

Back to our example $e \triangleq ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

	$(\hat{C}_e, \hat{\rho}_e)$	$(\hat{C}'_e, \hat{\rho}'_e)$	$(\hat{C}''_e, \hat{\rho}''_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
3	$\emptyset$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
4	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
y	$\emptyset$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$

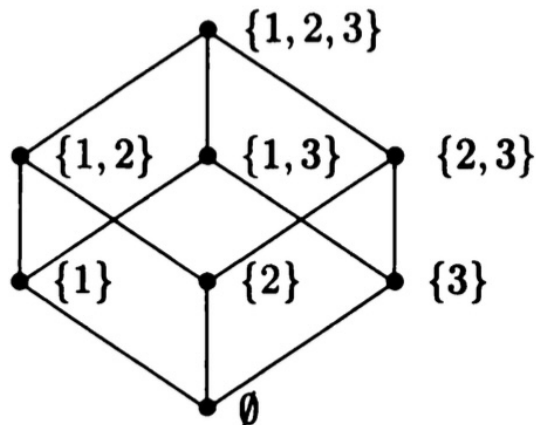
$$(\hat{C}'_e, \hat{\rho}'_e) \sqsubseteq (\hat{C}_e, \hat{\rho}_e)$$

Moore family

model intersection
property

A **Moore family** is a subset Y of a complete lattice $L = (L, \subseteq)$ that is closed under greatest lower bounds: $(\cap Y') \in Y$ for all $Y' \subseteq Y$

- It follows that a Moore family always contains a **least element** $\cap Y$, and a **greatest element** $\cap \emptyset$, which equals the greatest element $\top$ from L
- A Moore family is never empty



The complete lattice $(\wp(\{1,2,3\}), \subseteq)$

The subsets:

- $\{\{2\}, \{1,2\}, \{2,3\}, \{1,2,3\}\}$ and $\{\emptyset, \{1,2,3\}\}$ are both Moore families, while
- $\{\{1\}, \{2\}\}$ and $\{\emptyset, \{1\}, \{2\}, \{1,2\}\}$ are not

$\{1,2\} \cap \{2,3\} = \{2\} \in S \dots$
 $\cap \emptyset = \{1,2,3\} \in S$

$\emptyset \cap \{1,2,3\} = \emptyset \in S$ and
 $\cap \emptyset = \{1,2,3\} \in S$

$\{1\} \cap \{2\} = \emptyset \notin S$

$\cap \emptyset = \{1,2,3\} \notin S$

Existence of least CFA

The set of acceptable analyses is a complete lattice closed under intersection and it forms a **Moore family**

In particular, the set of acceptable analyses has a **least element**

Theorem

For all $ie \in \mathbf{IExp}$ the set $\{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$ is a **Moore family**

Proof idea

By **coinduction** on ie : we assume that $\forall i \in I : (\hat{C}_i, \hat{\rho}_i) \models ie$ and that $\sqcap_i (\hat{C}_i, \hat{\rho}_i) \models ie$

The least CFA is obtained by intersection

Existence

Theorem

For all $ie \in \mathbf{IExp}$ the set $\{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$ is a Moore family

taking the intersection over an empty set yields a valid analysis

Corollaries:

- all intermediate expressions admit a CFA: let $Y' = \emptyset$, then $(\sqcap Y') \in \{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$. So there is at least an analysis of ie
- all intermediate expressions have a least CFA: let $Y' = \{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$, then $(\sqcap Y') \in \{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$. So it will also be an analysis of ie . Clearly $(\sqcap Y') \sqsubseteq (\hat{C}, \hat{\rho})$ for all other analyses $(\hat{C}, \hat{\rho})$ of ie so it is the least analysis result.

Towards the least solution

Back to our example $e \triangleq ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

The last two are different solutions w.r.t. the previously introduced ones

	$(\hat{C}_e, \hat{\rho}_e)$	$(\hat{C}'_e, \hat{\rho}'_e)$	$(\hat{C}''_e, \hat{\rho}''_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1\}$
3	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } y \Rightarrow y^3\}$
4	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
y	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } y \Rightarrow y^3\}$

- $(\hat{C}'_e, \hat{\rho}'_e)$ and $(\hat{C}''_e, \hat{\rho}''_e)$ are acceptable, but neither is the least solution
- Their pointwise “intersection” $(\hat{C}'_e \sqcap \hat{C}''_e, \hat{\rho}'_e \sqcap \hat{\rho}''_e) = (\hat{C}_e, \hat{\rho}_e)$ is still a solution
- It is smaller, hence more precise, and is the **least acceptable solution**

Coinduction vs Induction

If acceptability were defined **inductively** with an acceptability relation $\models'$ then there exists an expression e_* such that acceptable analyses are not closed under intersection

Idea:

- If there are circular dependencies, it cannot resolve anything and may therefore return an empty solution
- Inductive definitions are too strict: they exclude valid over-approximations

Coinduction vs Induction

If acceptability were defined **inductively** with an acceptability relation $\models'$ then there exists an expression e_* such that acceptable analyses are not closed under intersection

Proposition

There exists $e_* \in \mathbf{Exp}$ such that the set $\{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models' e_*\}$ is **not** a Moore family

Proof idea: Use a very simple but “tricky” program and examine its CFA solutions:

- with the greatest fixed point $\rightarrow$ everything is fine
- with the least fixed point $\rightarrow$ we get a “bad” set ($\emptyset$)

We conclude it is not a Moore family

How to compute the least solution?

Least solutions (w.r.t. $\sqsubseteq$) exist, but how can we compute them?

Naive approach:

- 1.enumerate all candidate solutions
- 2.determine whether they are indeed solutions,
- 3.if so, take the **greatest lower bound** w.r.t. the others found so far

How to compute the least solution?

Least solutions (w.r.t. $\sqsubseteq$) exist, but how can we compute them?

Alternative:

1. derive a finite **set of constraints** from the program, such as the ones of the form $lhs \sqsubseteq rhs$,
2. take the **least solution** to this system of constraints (using fixpoint computation)

First, we will introduce a more computationally oriented specification: $\models_s$

Syntax directed specification

Syntax directed CFA

Syntax directed specification

Look for a more computationally oriented specification

Goal: analyze each function body *at most once* rather than each time the function could be applied

Idea: one way to obtain this is to analyse each function body *exactly once*, at its definition

This is done in the so-called **syntax directed O-CFA analysis**:

- acceptability relation: $\models_s$
- judgments: $(\hat{C}, \hat{\rho}) \models_s e_*$

Since semantic correctness was dealt with for the previous CFA there is no longer any need to consider intermediate expressions

Syntax directed specification

If we want to analyze a given expression e_* , we reformulate the specification of $(\hat{C}, \hat{\rho}) \models e_*$ into the more computationally oriented specification $(\hat{C}, \hat{\rho}) \models_s e_*$ such that:

$$(\hat{C}, \hat{\rho}) \models_s e_* \Rightarrow (\hat{C}, \hat{\rho}) \models e_* \text{ (if restricted to program terms)}$$

i.e., $\models_s$ is a safe approximation of $\models$: all solutions accepted by $\models_s$ are also valid for $\models$

We need to focus on solutions satisfying some additional properties (e.g., that only program fragments of e_* appear in the range of $\hat{C}$ and $\hat{\rho}$)

Clause for functions

$$[fn] (\hat{C}, \hat{\rho}) \models_s (fn\ x \Rightarrow e_0)^\ell \text{ iff } \{fn\ x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge (\hat{C}, \hat{\rho}) \models_s e_0$$

NEW: Function body e_0 is now analyzed at definition time, even if the function is never applied

Example

$$e \triangleq (fn\ x \Rightarrow x^1)^2$$

$$\hat{C}(2) \supseteq \{fn\ x \Rightarrow x^1\} \text{ and } \hat{\rho}(x) \subseteq \hat{C}(1)$$

Clause for functions (cont.)

$$[fn] (\hat{C}, \hat{\rho}) \models_s (fn\ x \Rightarrow e_0)^\ell \text{ iff } \{fn\ x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge$$

$$(\hat{C}, \hat{\rho}) \models_s e_0$$

Example

$$e \triangleq (fn\ x \Rightarrow ((fn\ y \Rightarrow y^2)^3\ x^4)^5)^6$$

$$(\hat{C}, \hat{\rho}) \models_s e \text{ iff } \{fn\ x \Rightarrow ((fn\ y \Rightarrow y^2)^3\ x^4)^5\} \subseteq \hat{C}(2) \wedge$$

$$(\hat{C}, \hat{\rho}) \models_s ((fn\ y \Rightarrow y^2)^3\ x^4)^5 \text{ iff } (\hat{C}, \hat{\rho}) \models_s (fn\ y \Rightarrow y^2)^3 \wedge (\hat{C}, \hat{\rho}) \models_s x^4 \wedge$$

$$(\hat{C}, \hat{\rho}) \models_s (fn\ y \Rightarrow y^2)^3 \text{ iff } \{fn\ y \Rightarrow y^2\} \subseteq \hat{C}(3) \wedge (\hat{C}, \hat{\rho}) \models_s y^2$$

$$(\hat{C}, \hat{\rho}) \models_s x^4 \text{ iff } \hat{\rho}(x) \subseteq \hat{C}(4) \quad (\hat{C}, \hat{\rho}) \models_s y^2 \text{ iff } \hat{\rho}(y) \subseteq \hat{C}(2)$$

$$\{fn\ y \Rightarrow y^2\} \in \hat{C}(3) : \hat{C}(4) \subseteq \hat{\rho}(y) \wedge \hat{C}(2) \subseteq \hat{C}(5)$$

These additional constraints are not required in $\models$, where only reachable behaviors matter

Clause for recursive functions

$$[fun] (\hat{C}, \hat{\rho}) \models_s (\text{fun } f \ x \Rightarrow e_0)^\ell \text{ iff } \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge$$

$$(\hat{C}, \hat{\rho}) \models_s e_0$$
$$\{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f)$$

NEW:

- **Function body** e_0 is now analysed at definition time, even if the function is never applied
- checks on the **recursive binding** for [fun] is included: self-binding is enforced immediately

See **Example 2** next

Clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models_s (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2}$$

NEW: The body constraints have already been generated by [fn]

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$

$$(\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell))$$

All other rules are unchanged: just replace $\models$ with $\models_s$

See **Example 2** next

Syntax directed CFA

- [con] $(\hat{C}, \hat{\rho}) \models_s c^\ell$ always
- [var] $(\hat{C}, \hat{\rho}) \models_s x^\ell$ iff $\hat{\rho}(x) \subseteq \hat{C}(\ell)$
- [fn] $(\hat{C}, \hat{\rho}) \models_s (\text{fn } x \Rightarrow e_0)^\ell$
iff $\{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge$
 $(\hat{C}, \hat{\rho}) \models_s e_0$
- [fun] $(\hat{C}, \hat{\rho}) \models_s (\text{fun } f \ x \Rightarrow e_0)^\ell$
iff $\{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge$
 $(\hat{C}, \hat{\rho}) \models_s e_0 \wedge \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f)$
- [app] $(\hat{C}, \hat{\rho}) \models_s (t_1^{\ell_1} t_2^{\ell_2})^\ell$
iff $(\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2} \wedge$
 $(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$
 $(\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell))$
- [if] $(\hat{C}, \hat{\rho}) \models_s (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell$
iff $(\hat{C}, \hat{\rho}) \models_s t_0^{\ell_0} \wedge$
 $(\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2} \wedge$
 $\hat{C}(\ell_1) \subseteq \hat{C}(\ell) \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$
- [let] $(\hat{C}, \hat{\rho}) \models_s (\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell$
iff $(\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2} \wedge$
 $\hat{C}(\ell_1) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$
- [op] $(\hat{C}, \hat{\rho}) \models_s (t_1^{\ell_1} \text{ op } t_2^{\ell_2})^\ell$ iff $(\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2}$

Back to Example 2

$\text{loop} \triangleq (\text{let } g = (\text{fun } f \text{ } x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)^5 \text{ in } (g^6 (\text{fn } z \Rightarrow z^7)^8)^9)^{10}$

$\hat{C}_{lp}(1) = \{f\}$	$\hat{C}_{lp}(6) = \{f\}$	$\hat{\rho}_{lp}(f) = \{f\}$
$\hat{C}_{lp}(2) = \emptyset$	$\hat{C}_{lp}(7) = \emptyset$	$\hat{\rho}_{lp}(g) = \{f\}$
$\hat{C}_{lp}(3) = \{\text{id}_y\}$	$\hat{C}_{lp}(8) = \{\text{id}_z\}$	$\hat{\rho}_{lp}(x) = \{\text{id}_y, \text{id}_z\}$
$\hat{C}_{lp}(4) = \emptyset$	$\hat{C}_{lp}(9) = \emptyset$	$\hat{\rho}_{lp}(y) = \emptyset$
$\hat{C}_{lp}(5) = \{f\}$	$\hat{C}_{lp}(10) = \emptyset$	$\hat{\rho}_{lp}(z) = \emptyset$

$f \triangleq \text{fun } f \text{ } x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$
 $\text{id}_y \triangleq \text{fn } y \Rightarrow y^2$
 $\text{id}_z \triangleq \text{fn } z \Rightarrow z^7$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s \text{loop}$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s f^5 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s g^6 (\text{fn } z \Rightarrow z^7)^8$
 $\wedge \hat{C}_{lp}(5) \subseteq \hat{\rho}_{lp}(g) \wedge \hat{C}_{lp}(9) \subseteq \hat{C}_{lp}(10)$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s f^5$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (f^1 (\text{fn } y \Rightarrow y^2)^3)^4 \wedge \{f\} \subseteq \hat{C}_{lp}(5) \wedge \{f\} \subseteq \hat{\rho}_{lp}(x)$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s f^1 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (\text{id}_y)^3$
 $\wedge \hat{C}_{lp}(3) \subseteq \hat{\rho}_{lp}(x) \wedge \hat{C}_{lp}(4) \subseteq \hat{C}_{lp}(4)$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (\text{id}_y)^3$ iff $\{\text{id}_y\} \subseteq \hat{C}_{lp}(3) \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s y^2$

Back to Example 2

$\text{loop} \triangleq (\text{let } g = (\text{fun } f \text{ } x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)^5 \text{ in } (g^6 (\text{fn } z \Rightarrow z^7)^8)^9)^{10}$

$\hat{C}_{lp}(1) = \{f\}$	$\hat{C}_{lp}(6) = \{f\}$	$\hat{\rho}_{lp}(f) = \{f\}$
$\hat{C}_{lp}(2) = \emptyset$	$\hat{C}_{lp}(7) = \emptyset$	$\hat{\rho}_{lp}(g) = \{f\}$
$\hat{C}_{lp}(3) = \{\text{id}_y\}$	$\hat{C}_{lp}(8) = \{\text{id}_z\}$	$\hat{\rho}_{lp}(x) = \{\text{id}_y, \text{id}_z\}$
$\hat{C}_{lp}(4) = \emptyset$	$\hat{C}_{lp}(9) = \emptyset$	$\hat{\rho}_{lp}(y) = \emptyset$
$\hat{C}_{lp}(5) = \{f\}$	$\hat{C}_{lp}(10) = \emptyset$	$\hat{\rho}_{lp}(z) = \emptyset$

$f \triangleq \text{fun } f \text{ } x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$
 $\text{id}_y \triangleq \text{fn } y \Rightarrow y^2$
 $\text{id}_z \triangleq \text{fn } z \Rightarrow z^7$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s \text{loop}$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s f^5 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s g^6 (\text{fn } z \Rightarrow z^7)^8$
 $\wedge \hat{C}_{lp}(5) \subseteq \hat{\rho}_{lp}(g) \wedge \hat{C}_{lp}(9) \subseteq \hat{C}_{lp}(10)$

using [app] and
that $\hat{C}(6) = \{f\}$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s g^6 (\text{fn } z \Rightarrow z^7)^8$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models g^6 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (\text{id}_z)^8$
 $\wedge \hat{C}_{lp}(8) \subseteq \hat{\rho}_{lp}(x) \wedge \hat{C}_{lp}(4) \subseteq \hat{C}_{lp}(9)$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (\text{id}_z)^8$ iff $\{\text{id}_z\} \subseteq \hat{C}_{lp}(8) \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s z^7$

Immediate

Back to Example 2

loop $\triangleq$ (let g = (fun f x => (f¹ (fn y => y²)³)⁴)⁵
in (g⁶ (fn z => z⁷)⁸)⁹)¹⁰

We have proved that $(\hat{C}_{lp}, \hat{\rho}_{lp})$ is an acceptable guess for $\models_s$

$\hat{C}_{lp}(1)$	=	{f}	$\hat{C}_{lp}(6)$	=	{f}	$\hat{\rho}_{lp}(f)$	=	{f}
$\hat{C}_{lp}(2)$	=	$\emptyset$	$\hat{C}_{lp}(7)$	=	$\emptyset$	$\hat{\rho}_{lp}(g)$	=	{f}
$\hat{C}_{lp}(3)$	=	{id _y }	$\hat{C}_{lp}(8)$	=	{id _z }	$\hat{\rho}_{lp}(x)$	=	{id _y , id _z }
$\hat{C}_{lp}(4)$	=	$\emptyset$	$\hat{C}_{lp}(9)$	=	$\emptyset$	$\hat{\rho}_{lp}(y)$	=	$\emptyset$
$\hat{C}_{lp}(5)$	=	{f}	$\hat{C}_{lp}(10)$	=	$\emptyset$	$\hat{\rho}_{lp}(z)$	=	$\emptyset$

$f \triangleq \text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$
 $\text{id}_y \triangleq \text{fn } y \Rightarrow y^2$
 $\text{id}_z \triangleq \text{fn } z \Rightarrow z^7$

No circular dependency => no need of coinduction

Observation

We can take $\models_s$ as the **largest relation satisfying the specification**

However, given the syntax directed nature of the specification (the relation is fully determined by the structure of expressions), there is in fact **only one** relation that satisfies the specification

unique

Hence it would be technically correct, but intuitively misleading, to claim that we take the **least relation that satisfies the specification**, because it suggests a choice among many relations

Before: many acceptable relations $\rightarrow$ take the least

Now: only one relation $\rightarrow$ no minimisation needed

Whether or not $\models_s$ is defined inductively or conductively, the same relation is defined. This is typical of syntax-directed definitions

Exercise

Let $\models'_s$ and $\models''_s$ be two relations satisfying the syntax-directed CFA clauses. Show, by *structural induction* on e that

$$(\hat{C}, \hat{\rho}) \models'_s e \text{ iff } (\hat{C}, \hat{\rho}) \models''_s e$$

[Case c^ℓ]: trivial

[Case x^ℓ]: $(\hat{C}, \hat{\rho}) \models'_s x^\ell$ iff $\hat{\rho}(x) \subseteq \hat{C}(\ell)$ and $(\hat{C}, \hat{\rho}) \models''_s x^\ell$ iff $\hat{\rho}(x) \subseteq \hat{C}(\ell)$

Both relations satisfy the same clause on $(\hat{C}, \hat{\rho})$, so they are equivalent.

[Case $(f\ x \Rightarrow t_0^{\ell_0})^\ell$]: $(\hat{C}, \hat{\rho}) \models'_s (f\ x \Rightarrow t_0^{\ell_0})^\ell$ iff $\{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge (\hat{C}, \hat{\rho}) \models'_s e_0$,
and $(\hat{C}, \hat{\rho}) \models''_s (f\ x \Rightarrow t_0^{\ell_0})^\ell$ iff $\{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge (\hat{C}, \hat{\rho}) \models''_s e_0$

In each case, both $\models'_s$ and $\models''_s$ satisfy the same defining clause. Hence their equivalence reduces to the equivalence of the premises, which follows by the induction hypothesis $((\hat{C}, \hat{\rho}) \models'_s t_0^{\ell_0} \text{ iff } (\hat{C}, \hat{\rho}) \models''_s t_0^{\ell_0})$.

The remaining cases are analogous.

Observation on the exercise

If $\models'$ and $\models''$ are two relations satisfying the **non** syntax-directed CFA clauses. Then, we cannot establish that

$$(\hat{C}, \hat{\rho}) \models' e \text{ iff } (\hat{C}, \hat{\rho}) \models'' e$$

$a^3 \triangleq (0^1 0^2)^3$: and suppose to have that

- $(\text{fn } x \Rightarrow a^4) \in \hat{C}(1)$,
- $\hat{C}(2) = \hat{C}(2) = \rho(x) = \emptyset$

The function in $\hat{C}(1)$ introduces a dependency outside the syntactic structure

$$(\hat{C}, \hat{\rho}) \models' a^3 \text{ iff } (\hat{C}, \hat{\rho}) \models' 0^1 \wedge (\hat{C}, \hat{\rho}) \models' 0^2 \wedge (\hat{C}, \hat{\rho}) \models' a^4 \wedge \hat{C}(2) \subseteq \rho(x) \wedge \hat{C}(4) \subseteq \hat{C}(3)$$

The only critical point is that $(\hat{C}, \hat{\rho}) \models' a^3$ iff $(\hat{C}, \hat{\rho}) \models' a^4$

This equivalence holds true whether or not you include 'a' in the relation, i.e., $(\hat{C}, \hat{\rho}) \models' a^3$ either is true or is false.

Therefore, we can have two different relations that satisfy the same clauses: one that includes and one that does not.

Preservation of solutions

The specification of the analysis uses **potentially infinite value spaces** although this is not really necessary

We can easily restrict ourselves to entities occurring in the original expression and this forms the basis for relating the results of the analysis of syntax directed CFA to those of the previous CFA

We obtain finite sets

Lab_{*} $\subseteq$ **Lab**: finite set of labels occurring in e_* ,

Var_{*} $\subseteq$ **Var**: finite set of variables occurring in e_*

Term_{*} $\subseteq$ **Term**: finite set of subterms occurring in e_*

Preservation of solutions

We introduce the solution $(\hat{C}^\top, \hat{\rho}^\top)$

the largest possible solution
within the bounds of the
expression

$$\hat{C}^\top(\ell) = \begin{cases} \emptyset & \text{if } \ell \notin \mathbf{Lab}_* \\ \mathbf{Term}_* & \text{if } \ell \in \mathbf{Lab}_* \end{cases} \quad \hat{\rho}^\top(x) = \begin{cases} \emptyset & \text{if } x \notin \mathbf{Var}_* \\ \mathbf{Term}_* & \text{if } x \in \mathbf{Var}_* \end{cases}$$

$(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}^\top, \hat{\rho}^\top)$: intuitively, $(\hat{C}, \hat{\rho})$ is concerned **only with subterms occurring in the expression e_***

We can directly reformulate the CFA in terms of restricted sets

$$(\hat{C}, \hat{\rho}) \in \widehat{\mathbf{Cache}_*} \times \widehat{\mathbf{Env}_*}$$

$$\widehat{\mathbf{Cache}_*} : \mathbf{Lab}_* \rightarrow \widehat{\mathbf{Val}_*}$$

$$\widehat{\mathbf{Env}_*} : \mathbf{Var}_* \rightarrow \widehat{\mathbf{Val}_*}$$

$$\widehat{\mathbf{Val}_*} = \wp(\mathbf{Term}_*)$$

Why syntax-directed CFA is correct

check when needed

Two views of the analysis

- Original CFA $\models$: checks correctness during the simulation of the evaluation, i.e., when analysing function application
- Syntax-directed CFA $\models_s$: analyzes all subterms upfront, so when we reach a call site, everything has already been analysed

As a consequence:

check in advance: nothing missing at call time

$$(\hat{C}, \hat{\rho}) \models_s e_* \Rightarrow (\hat{C}, \hat{\rho}) \models e_* \text{ (if restricted to program terms)}$$

Preservation of solutions

- We can prove that all the solutions to $\models_s e_*$ that are "less than" $(\hat{C}^\top, \hat{\rho}^\top)$ are also solutions to $\models e_*$
- Syntax-directed CFA is a **sound approximation** of the original CFA when restricted to program terms

boundedness condition

Proposition

If $(\hat{C}, \hat{\rho}) \models_s e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^\top, \hat{\rho}_*^\top)$ then $(\hat{C}, \hat{\rho}) \models e_*$

Idea: syntax-directed CFA already enforces what the original CFA requires

Proof idea:

- Restrict to program terms and use coinduction
- Show that every syntax-directed clause implies the original one
- Therefore, the solution also holds for the original CFA

Preservation of solutions

Proposition

If $(\hat{C}, \hat{\rho}) \models_s e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}^\top, \hat{\rho}^\top)$ then $(\hat{C}, \hat{\rho}) \models e_*$

Proof sketch:

1. Restrict to program terms $\mathbf{Exp}_*$, the set of all sub-expressions occurring in e_*
2. By syntax-directedness: $(\hat{C}, \hat{\rho}) \models_s e_* \Rightarrow \forall e \in \mathbf{Exp}_* : (\hat{C}, \hat{\rho}) \models_s e$
3. By coinduction, show that every syntax-directed clause (its *rhs*) implies the original one. Intuitively:
 - [con], [var], [if], [let], [op]: immediate, since identical.
 - [fn], [fun]: syntax-directed rules are stronger than the original ones.
 - [app]: $\models_s$ already enforces all constraints required by $\models$ (function bodies are already analyzed by (2)).

So, $\models_s$ captures all semantically required flows on reachable terms

Does the converse hold?

$(\hat{C}, \hat{\rho}) \models_s e_* \stackrel{?}{\Leftarrow} (\hat{C}, \hat{\rho}) \models e_*$ (if restricted to program terms)

X No

f never applied
program evaluates to 0

$\text{loop} \triangleq \text{let } f = \text{fn } x \Rightarrow (x \ x) \ (* \text{ problematic body } *)$
in 0

- Original CFA: checks only when needed
- Syntax-directed CFA: checks all bodies
- Unreachable code is ignored by $\models$, but not by $\models_s$
- $\models_s \subseteq \models$: Syntax-directed CFA is more restrictive

Existence

Theorem

$\{(\hat{C}, \hat{\rho}) \in (\widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*) \mid (\hat{C}, \hat{\rho}) \models_s e\}$ is a Moore family

Corollaries:

- Each expression e_* admits a CFA that is "less than" $(\hat{C}^\top, \hat{\rho}^\top)$
- Each expression e_* has a "least" CFA that is "less than" $(\hat{C}^\top, \hat{\rho}^\top)$

This means that the properties obtained for the original analysis holds for the syntax-directed one

Existence

Theorem

$\{(\hat{C}, \hat{\rho}) \in (\widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*) \mid (\hat{C}, \hat{\rho}) \models_s e\}$ is a **Moore family**

Proof:

$(\hat{C}_*^\top, \hat{\rho}_*^\top)$ is the greatest element of $\widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*$. It is immediate to show that:

(a) $(\hat{C}_*^\top, \hat{\rho}_*^\top) \models_s e$

(b) $(\hat{C}_1, \hat{\rho}_1) \models_s e$ and $(\hat{C}_2, \hat{\rho}_2) \models_s e$ then $((\hat{C}_1, \hat{\rho}_1) \cap (\hat{C}_2, \hat{\rho}_2)) \models_s e$

for all **subexpressions** e of e_* , by means of *structural induction* on e . Hence:

(a) and (b) also for $e = e_*$.

Next consider some

$$Y \subseteq \{(\hat{C}, \hat{\rho}) \in (\widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*) \mid (\hat{C}, \hat{\rho}) \models_s e\}$$

and note that one can write

$Y = \{(\hat{C}_i, \hat{\rho}_i) \mid i \in \{1, \dots, n\}\}$ for some $n \geq 0$ since $(\widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*)$ is finite. So

$$\sqcap Y \in \{(\hat{C}_i, \hat{\rho}_i) \mid i \in \{1, \dots, n\}\}$$

follows from (a) and (b), because

$$\sqcap Y = (\hat{C}_*^\top, \hat{\rho}_*^\top) \sqcap (\hat{C}_1, \hat{\rho}_1) \sqcap \dots \sqcap (\hat{C}_n, \hat{\rho}_n)$$

Exercise 1

Modify the original Control Flow Analysis to take account of the left to right evaluation order imposed by a call-by-value semantics:

- in the clause [app] there is no need to analyse the operand if the operator cannot produced any closures

Clause for function application

$$\begin{aligned} [app] \ (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } & (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge \\ & (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ & \quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\ & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge \\ & (\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ & \quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\ & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge \\ & \quad \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f)) \end{aligned}$$

New clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge$$

guard operand analysis

$$(\exists (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) \Rightarrow (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}) \wedge$$

...

If the only abstract values are only abstractions, we can use also

$$((\hat{C}(\ell_1) \cap \widehat{Val}) \neq \emptyset \Rightarrow (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}) \wedge$$

Example: Try to find a program where the modified analysis accepts analysis results rejected by the original analysis

$e \triangleq 4 \text{ (fn } y \Rightarrow y)$: the empty analysis is acceptable for the modified analysis but not for the original one

Reachability analysis

- The syntax directed analysis analyzes each subexpression of e_* “exactly once”, rather than “at most once” as really called for
- This may introduce constraints from code that is never executed
- How to amend this?

Goal:

- only expressions that are reachable will actually be analysed, and
- the same expression is analysed “at most once”

Idea:

1. modify the analysis to track the reachability of subexpressions explicitly
2. modify it such that subexpressions are analysed “at most once”

Reachability analysis

How?

By introducing a new abstract reachability component

$$\hat{R} \in \mathbf{Reach} : \mathbf{Lab} \rightarrow \wp(\{\mathbf{on}\})$$

and modifying the syntax directed analysis

$$(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e$$

$\{\mathbf{on}\} \subseteq \hat{R}(\ell)$ means: the expression at label ℓ is reachable

- $\hat{R}(\ell) = \emptyset$: the program point ℓ is not reachable (t_ℓ must be analysed)
- $\{\mathbf{on}\} \subseteq \hat{R}(\ell_0)$: the program point ℓ is reachable
- $(\text{fn } x \Rightarrow t_0^{\ell_0})^\ell$ has $\{\mathbf{on}\} \subseteq \hat{R}(\ell_0)$ iff the function is indeed applied somewhere
- A function body is analyzed only if it is reachable: the "recursive call"
 $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_0^{\ell_0}$ is performed if and only if $\{\mathbf{on}\} \subseteq \hat{R}(\ell_0)$
- Hence, function bodies are analyzed “at most once”

Clause for functions

$$[fn] (\hat{C}, \hat{\rho}, \hat{R}) \models'_s (\text{fn } x \Rightarrow e_0)^\ell \text{ iff } \{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge$$
$$(\{\mathbf{on}\} \subseteq \hat{R}(\ell) \Rightarrow \wedge (\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_0)$$

NOW: Function body e_0 is analysed at definition time, but only if the function is applied somewhere

Clause for variables

$$[var] (\hat{C}, \hat{\rho}, \hat{R}) \models'_s x^\ell \text{ iff } (\{\mathbf{on}\} \subseteq \hat{R} \Rightarrow \rho(x) \subseteq \hat{C}(\ell))$$

NOW: x is analyzed only if reachable

Clause for function application

$$[app] (\hat{C}, \hat{\rho}, \hat{R}) \models'_s (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_2^{\ell_2}$$

NEW: If the function possibly occurs in an application, then this means it is reachable

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ (\{\mathbf{on}\} \subseteq \hat{R}(\ell) \Rightarrow \{\mathbf{on}\} \subseteq \hat{R}(\ell_0)) \wedge \\ (\{\mathbf{on}\} \subseteq \hat{R}(\ell) \Rightarrow \hat{C}(\ell_2) \subseteq \hat{\rho}(x)) \wedge \\ (\{\mathbf{on}\} \subseteq \hat{R}(\ell) \Rightarrow \hat{C}(\ell_0) \subseteq \hat{C}(\ell))) \wedge \dots$$

All other rules are unchanged: just replace $\models_s$ with $\models'_s$

Example

$$e_* \triangleq (\text{fn } x \Rightarrow ((\text{fn } y \Rightarrow y^1)^2 x^3)^4)^5$$

We start knowing that $\hat{R}(\ell_*) = \hat{R}(5) = \emptyset$

$$(\hat{C}, \hat{\rho}, \hat{R}) \vdash'_s e_*$$

As a consequence, $\{\text{fn } x \Rightarrow ((\text{fn } y \Rightarrow y^1)^2 x^3)^4\} \subseteq \hat{C}(5)$, but we cannot analyse the body with label 4, since $\hat{R}(5) = \emptyset$

Example

$$e_* \triangleq ((\text{fn } x \Rightarrow ((\text{fn } y \Rightarrow y^1)^2 x^3)^4)^5 \textcolor{red}{7})^6$$

$$(\hat{C}, \hat{\rho}, \hat{R}) \vDash'_s e_*$$

As a consequence, $\{\text{fn } x \Rightarrow ((\text{fn } y \Rightarrow y^1)^2 x^3)^4\} \subseteq \hat{C}(5)$

This triggers the propagation in the [app] clause that enforces $\{\mathbf{on}\} \subseteq \hat{R}(4)$, so allowing to analyse the body of the function

Reachability

Proposition

If $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_*^{\ell_*}$ and $\{\mathbf{on}\} \subseteq \hat{R}(\ell_*)$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^T, \hat{\rho}_*^T)$ then $(\hat{C}, \hat{\rho}) \models t_*^{\ell_*}$

Proof sketch:

1. Restrict to program terms, $\mathbf{Exp}_*$, the set of all reachable subexpressions occurring in e_*
2. By syntax-directedness: $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_*^{\ell_*} \Rightarrow \forall e \in \mathbf{Exp}_* : (\{\mathbf{on}\} \subseteq \hat{R}(\ell_*) \Rightarrow (\hat{C}, \hat{\rho}) \models_s e)$
3. By coinduction, show that every syntax-directed clause (its *rhs*) implies the original one.

Intuitively:

- [con], [var], [if], [let], [op]: immediate, since identical.
- [fn],[fun]: syntax-directed rules with the additional constraint on $\hat{R}$ are equivalent to the original ones, on reachable terms.
- [app]: $\models_s$ enforces all constraints required by $\models$ on reachable terms, and propagates reachability information $\hat{R}$.

So, $\models'_s$ captures all semantically required flows on reachable terms, provided that $\hat{R}$ correctly over-approximates reachability

Reachability

Proposition

If $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_*^{\ell_*}$ and $\{\mathbf{on}\} \subseteq \hat{R}(\ell_*)$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^T, \hat{\rho}_*^T)$ then $(\hat{C}, \hat{\rho}) \models t_*^{\ell_*}$

Does the converse hold?

1. $(\hat{C}, \hat{\rho}) \models e_*$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$
 2. $(\hat{C}, \hat{\rho}) \models e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^T, \hat{\rho}_*^T)$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$
1. does not hold:
- the original CFA can introduce and treat consistently “spurious” abstract values, without loosing acceptability, since it provides an over-approximation
 - the reachability CFA, instead, even forcing to **on** the “spurious” value, cannot analyze the abstraction (and its body), since it do not occur in e_*
2. $(\hat{C}, \hat{\rho}) \models e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^T, \hat{\rho}_*^T)$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$

Reachability

Proposition

If $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_*^{\ell_*}$ and $\{\mathbf{on}\} \subseteq \hat{R}(\ell_*)$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^{\top}, \hat{\rho}_*^{\top})$ then $(\hat{C}, \hat{\rho}) \models t_*^{\ell_*}$

Does the converse hold?

1. $(\hat{C}, \hat{\rho}) \models e_*$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$
2. $(\hat{C}, \hat{\rho}) \models e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^{\top}, \hat{\rho}_*^{\top})$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$
2. Since $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^{\top}, \hat{\rho}_*^{\top})$ then no “spurious” value can be added. In this case the bodies analyzed in the two analyses are the same, provided a suitable $\hat{R}$ is provided